

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	25/10/2025
	Nombre: Kevin Damian	

Actividad 2: Despliegue del aplicativo para gestión financiera en multi-contenedor con Docker Compose

Link para el repositorio (he enviado invitación al repositorio así como en adjunto las fuentes):

<https://github.com/FadedGuy/MastersDevOps/tree/main/S2/Contenedores/Actividad1>

Introducción

El presente informe documenta la actividad practica que tiene como objetivo usar un docker-compose como archivo de configuración para definir el “stack” de la aplicación creada en la clase pasada y así poder levantar toda su infraestructura con un solo comando (servicios, volúmenes y redes).

Descripción y diseño de la solución

A modo de recordatorio, el entorno por desplegar constara de tres contenedores conectados mediante una red bridge de Docker. La separación se realizo por “capa”, es decir, cada imagen es responsable de una responsabilidad

- WordPress (PHP-FPM)

Representa la capa de la aplicación y lógica de negocio. Se construyo a partir de la imagen oficial de PHP-FPM a la cual se le agrego la descarga e instalación de WordPress y WordPress CLI. Esta configurada para recibir mediante variables de entorno o archivo de configuración los valores necesarios para conectarse y/o crear la base de datos.

- MySQL

Funciona como la base de datos. Se emplea la imagen oficial de MySQL, manteniendo su configuración por defecto salvo por pasar mediante variables de entorno la configuración de credenciales y nombre. Además, se le asocio un volumen para garantizar la durabilidad de los datos sin importar si el contenedor se destruye

- Nginx

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	25/10/2025
	Nombre: Kevin Damian	

Actúa como servidor HTTP y proxy inverso, reenviando las peticiones dinámicas al contenedor de WordPress, ya que PHP-FPM no soporta ser un servidor web. Para este se construyó una imagen personalizada con una configuración precargada del nginx.conf. Asegurando que esta lista desde el arranque, pero aun dejando la posibilidad a ser cambiada

Implementación

El siguiente docker-compose.yaml define el entorno mencionado anteriormente, incluyendo la configuración de los servicios necesarios para ejecutar WordPress junto con su base de datos MySQL. Además, se especifica la creación del volumen para la base de datos y una red interna única para la comunicación entre estos servicios. Dado que la imagen oficial de MySQL no incluye un mecanismo de healthcheck, se ha indicado uno que indica cuando el contenedor está listo para aceptar conexiones; algo esencial para WordPress. Asimismo, se declaran explícitamente todas las variables de entornos por la imagen de WordPress para permitir la configuración específica del proyecto.

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	25/10/2025
	Nombre: Kevin Damian	

```

name: "wordpress"
D> Run All Services
services:
  # Servicio de base de datos MySQL
  D> Run Service
  wp-db:
    # Imagen de MySQL 8.1
    image: mysql:8.1
    # Identificador del contenedor
    container_name: wp-db
    # Volumen persistente para los datos
    volumes:
      - wp-db-data:/var/lib/mysql
    # Variables de entorno para la configuracion de la bd
    environment:
      MYSQL_ROOT_PASSWORD: "rootpassword"
      MYSQL_DATABASE: "wordpress"
      MYSQL_USER: "wpuser"
      MYSQL_PASSWORD: "wppassword"
    # Red interna especifica para la comunicacion de servicios de este proyecto
    networks:
      - wp-network
    # Verifica que la BD esté lista antes de continuar
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -uroot -prootpassword | grep 'mysqld is alive'"]
      interval: 10s
      timeout: 10s
      retries: 5
      start_period: 20s

  # Servicio PHP-FPM con WordPress
  D> Run Service
  wp-php:
    # El build se hace desde la carpeta ./WordPress con el dockerfile de nombre Dockerfile
    build:
      context: ./WordPress
      dockerfile: Dockerfile
    # Nombre de la imagen generada por el build
    image: fadedguy/my-wordpress:latest
    container_name: wp-php
    # Variables de entorno para la configuracion de WP y su conexion de la db
    environment:
      DB_NAME: "wordpress"
      DB_USER: "wpuser"
      DB_PASSWORD: "wppassword"
      DB_HOST: "wp-db"
      WP_URL: "http://localhost:8080"
      WP_TITLE: "WordPressDockerCompose"
      WP_ADMIN_USER: "admin"
      WP_ADMIN_PASSWORD: "adminpassword"
      WP_ADMIN_EMAIL: "kevin.aceves@hotmail.com"
    # Espera a que wp-db pase el healthcheck
    depends_on:
      wp-db:
        condition: service_healthy
    networks:
      - wp-network

  # Servicio Nginx como proxy inverso
  D> Run Service
  wp-nginx:
    # El build se hace desde la carpeta ./Nginx con el dockerfile de nombre Dockerfile
    build:
      context: ./Nginx
      dockerfile: Dockerfile
    # Nombre de la imagen generada por el build
    image: fadedguy/my-nginx:latest
    container_name: wp-nginx
    # Mapear el puerto 80 del contenedor al puerto 8080 del host
    ports:
      - "8080:80"
    # Espera a que wp-php haya sido inicializado
    depends_on:
      - wp-php
    networks:
      - wp-network

volumes:
  wp-db-data:
    driver: local

networks:
  wp-network:
    driver: bridge

```

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	25/10/2025
	Nombre: Kevin Damian	

Despliegue

Para desplegar los servicios se corre el comando *docker compose up*, o *docker compose up --build* en caso de que queramos construir las imágenes, a continuación, un ejemplo de cada una.

docker compose up crea los 3 contenedores, así como el volumen y red en caso de no existir

```

PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1> docker compose up
[+] Running 3/3
  ✓ Container wp-db      Created                                0.0s
  ✓ Container wp-php      Created                                0.0s
  ✓ Container wp-nginx    Created                                0.0s
Attaching to wp-db, wp-nginx, wp-php
wp-db | 2025-10-26 00:43:38+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.1.0-1.el8 started.
wp-db | 2025-10-26 00:43:38+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
wp-db | 2025-10-26 00:43:38+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.1.0-1.el8 started.
wp-db | '/var/lib/mysql/mysql.sock' -> '/var/run/mysql/mysql.sock'
wp-db | 2025-10-26T00:43:39.358213Z 0 [System] [MY-015015] [Server] MySQL Server - start.
wp-db | 2025-10-26T00:43:39.809799Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future release
wp-db | . Please use SET GLOBAL host_cache_size=0 instead.
wp-db | 2025-10-26T00:43:39.816255Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.1.0) starting as process 1
wp-db | 2025-10-26T00:43:39.913788Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
wp-db | 2025-10-26T00:43:40.834189Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
wp-db | 2025-10-26T00:43:41.436951Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
wp-db | 2025-10-26T00:43:41.437052Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported
wp-db | for this channel.
wp-db | 2025-10-26T00:43:41.442568Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is acc
wp-db | essible to all OS users. Consider choosing a different directory.
wp-db | 2025-10-26T00:43:41.544592Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysq
wp-db | ld/mysqlx.sock
wp-db | 2025-10-26T00:43:41.545086Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.1.0' socket: '/var/run/mysqld/m
wp-php | [26-Oct-2025 00:43:44] NOTICE: [pool www] 'user' directive is ignored when FPM is not running as root
wp-php | [26-Oct-2025 00:43:44] NOTICE: [pool www] 'user' directive is ignored when FPM is not running as root
wp-php | [26-Oct-2025 00:43:44] NOTICE: [pool www] 'group' directive is ignored when FPM is not running as root
wp-php | [26-Oct-2025 00:43:44] NOTICE: [pool www] 'group' directive is ignored when FPM is not running as root
wp-php | [26-Oct-2025 00:43:44] NOTICE: fpm is running, pid 7
wp-php | [26-Oct-2025 00:43:44] NOTICE: ready to handle connections
wp-nginx | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
wp-nginx | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
wp-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
wp-nginx | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
wp-nginx | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
wp-nginx | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
wp-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
wp-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
wp-nginx | /docker-entrypoint.sh: Configuration complete; ready for start up
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: using the "epoll" event method
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: nginx/1.29.1
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14+deb12u1)
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: OS: Linux 5.15.167.4-microsoft-standard-WSL2
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker processes
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 28
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 29
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 30
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 31
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 32
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 33
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 34
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 35
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 36
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 37
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 38
wp-nginx | 2025/10/26 00:43:44 [notice] 1#1: start worker process 39

```

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	25/10/2025
	Nombre: Kevin Damian	

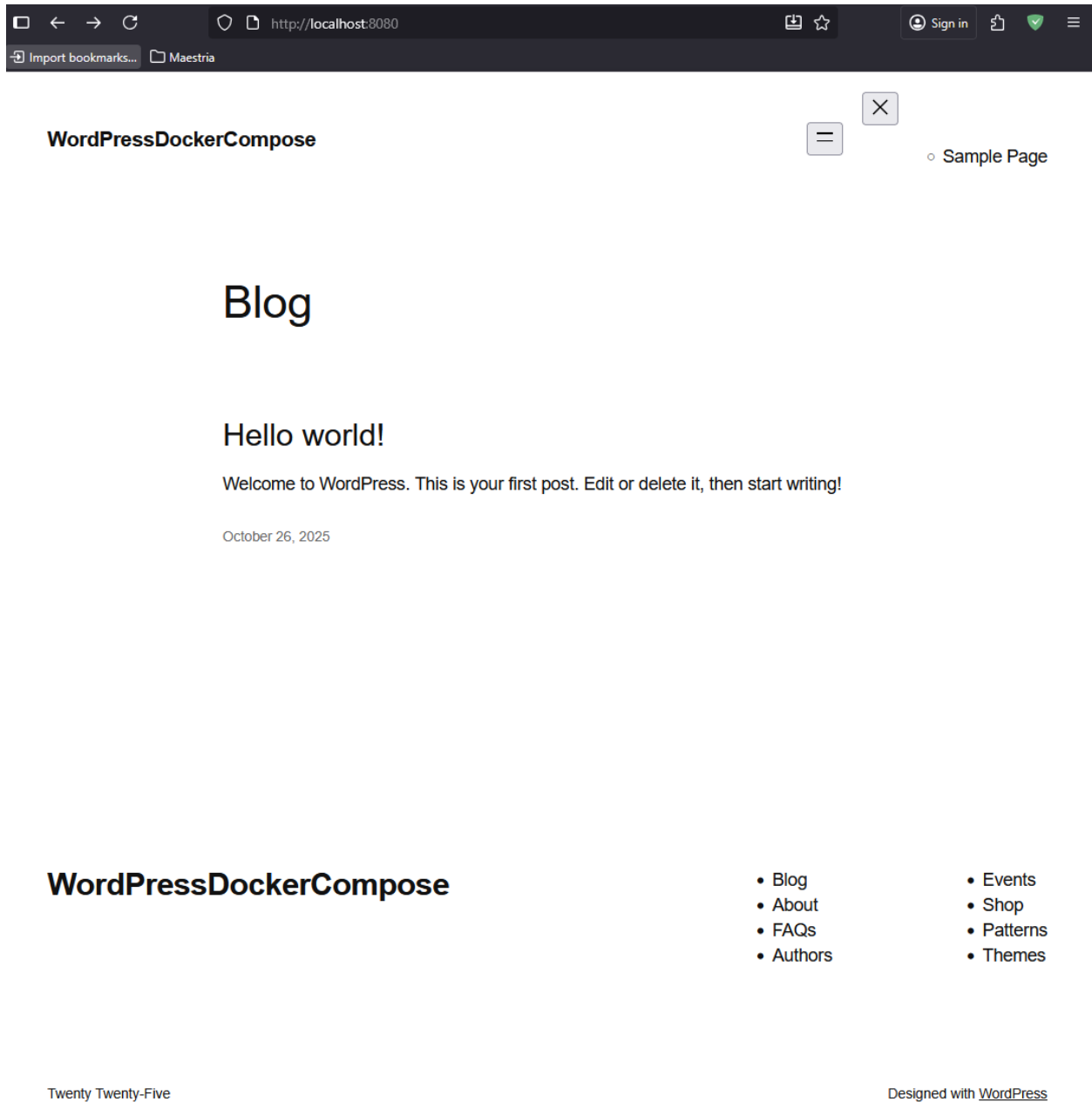
Por otro lado, *docker compose up --build*, hace lo mismo que la imagen anterior, pero este construye previamente las imágenes en caso de que hayan cambiado o simplemente se quieran regenerar.

```
PS D:\Documents\Projects\Prog\MastersDevOps\S2\Contenedores\Actividad1> docker compose up --build
Compose now can delegate build to bake for better performances
Just set COMPOSE_BAKE=true
[+] Building 7.4s (24/24) FINISHED                                docker:desktop-linux
=> [wp-php internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.21kB                                0.1s
=> [wp-php internal] load metadata for docker.io/library/php:8.2-fpm 0.1s
=> [wp-php auth] library/php:pull token for registry-1.docker.io    1.6s
=> [wp-php internal] load .dockerignore                             0.0s
=> => transferring context: 2B                                       0.1s
=> [wp-php 1/8] FROM docker.io/library/php:8.2-fpm@sha256:14b7774a36d424da12c1a138fad283142bdf297ebd625176dce04ea674781c6c 0.0s
=> => resolve docker.io/library/php:8.2-fpm@sha256:14b7774a36d424da12c1a138fad283142bdf297ebd625176dce04ea674781c6c 0.1s
=> [wp-php internal] load build context                             0.1s
=> => transferring context: 1.28kB                                    0.1s
=> CACHED [wp-php 2/8] RUN apt-get update && apt-get install -y     default-mysql-client libzip-dev libonig-dev libxml2-dev curl 0.0s
=> CACHED [wp-php 3/8] RUN mkdir -p /var/www && chown www-data:www-data /var/www 0.0s
=> CACHED [wp-php 4/8] RUN curl -sSL https://wordpress.org/latest.tar.gz | tar -xz -C /var/www && chown -R www-data:www-data /var/www/wordpress 0.0s
=> CACHED [wp-php 5/8] RUN curl -O https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar && chmod +x wp-cli.phar && mv wp 0.0s
=> CACHED [wp-php 6/8] COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh 0.0s
=> CACHED [wp-php 7/8] RUN chmod +x /usr/local/bin/docker-entrypoint.sh 0.0s
=> CACHED [wp-php 8/8] WORKDIR /var/www/wordpress 0.0s
=> [wp-php] exporting to image
=> => exporting layers
=> => exporting manifest sha256:6cab92c348ee68291dee1607ab31836b23eb42d321b653a5ba2a98bf21b1c26e 0.0s
=> => exporting config sha256:33ab219fd60606661b77e9f2429d186d4b668a8177b2380ab408712848f71b37 0.0s
=> => exporting attestation manifest sha256:d0bd81cd8bbd03982815c67a8a580ca0becbefb79a0c14a36db60aaca6667f58 0.1s
=> => exporting manifest list sha256:bd9ef4c26ab636e74b62061bfd9b3b29d9dad3d835f2483515e4e642acd9af0 0.1s
=> => naming to docker.io/fadedguy/my-wordpress:latest 0.0s
=> => unpacking to docker.io/fadedguy/my-wordpress:latest 0.1s
=> [wp-php] resolving provenance for metadata file 0.4s
=> [wp-nginx internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 343B 0.0s
=> [wp-nginx internal] load metadata for docker.io/library/nginx:latest 0.1s
=> [wp-nginx internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [wp-nginx internal] load build context 0.0s
=> => transferring context: 32B 0.0s
=> [wp-nginx 1/2] FROM docker.io/library/nginx:latest@sha256:75cad0ed4b9ea94413143f700eea1a71635806207ffb901b577c436a06ff4acd 0.9s
=> => resolve docker.io/library/nginx:latest@sha256:75cad0ed4b9ea94413143f700eea1a71635806207ffb901b577c436a06ff4acd 0.9s
=> [wp-nginx auth] library/nginx:pull token for registry-1.docker.io 0.0s
=> CACHED [wp-nginx 2/2] COPY nginx.conf /etc/nginx/conf.d/default.conf 0.0s
=> [wp-nginx] exporting to image 0.3s
=> => exporting layers 0.0s
=> => exporting manifest sha256:94660a7b7a663aef208b09e991aa4fa0afbed884cc76551178710249c158f39f 0.0s
=> => exporting config sha256:c3e4b2859b94ebc2b07a8fc9680d0fe3e5e5c479d621df84b869af3d1cd5fce4 0.0s
=> => exporting attestation manifest sha256:4aa2912cb8bb235962bc46a868667a8b0eb19b5dae5f0eef7e41cb876d3c81a5 0.1s
=> => exporting manifest list sha256:a98a3d8dd9dcacd1c432775fc3135b4bf0f081d2eda95ff5c6efa4756b9bf1de 0.0s
=> => naming to docker.io/fadedguy/my-nginx:latest 0.0s
=> => unpacking to docker.io/fadedguy/my-nginx:latest 0.0s
=> [wp-nginx] resolving provenance for metadata file 0.0s
[+] Running 5/5
✔ wp-nginx Built 0.0s
✔ wp-php Built 0.0s
✔ Container wp-db Created 0.0s
✔ Container wp-nginx Recreated 1.5s
✔ Container wp-php Recreated 0.7s
```

Una vez se encuentren funcionales los servicios, es posible probar que la aplicación se ha lanzado y configurado correctamente ingresando en el navegador a la dirección <http://localhost:8080>. A continuación, una captura de pantalla del resultado, mostrando la página inicial de WordPress ya

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	25/10/2025
	Nombre: Kevin Damian	

configurada.



Ahora para parar o bien destruir los contenedores, redes y volúmenes creados se tienen dos principales alternativas en este caso, la primera eliminara unicamente la red y servicios, la segunda eliminara los mismos asi como el volumen.

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	25/10/2025
	Nombre: Kevin Damian	

La primera se trata de *docker compose down*

```

PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1> docker compose down
[+] Running 4/4
✓ Container wp-nginx      Removed      0.0s
✓ Container wp-php        Removed      0.0s
✓ Container wp-db         Removed      0.0s
✓ Network wordpress_wp-network Removed      0.7s

```

La segunda es también *docker compose down -v*, con la bandera -v, que la cual indica que elimine los volúmenes que se crean con el stack

```

PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1> docker compose down -v
[+] Running 5/5
✓ Container wp-nginx      Removed      0.0s
✓ Container wp-php        Removed      0.0s
✓ Container wp-db         Removed      0.0s
✓ Volume wordpress_wp-db-data Removed      0.0s
✓ Network wordpress_wp-network Removed      0.7s

```

Conclusiones

La actividad permitió comprender y aplicar fundamentos de Docker Compose para el despliegue de aplicaciones compuestas de múltiples contenedores, volúmenes y redes. Mediante la definición de servicios, volúmenes y redes es posible hacer el proceso de despliegue y garantizar la reproducibilidad del entorno. La automatización del arranque, junto con el nuevo uso de *healthchecks* y variables de entorno permite obtener un entorno mas confiable, coherente y fácil de mantener.

además, se ha reforzado la importancia de Docker Compose como herramienta para el desarrollo y orquestación eficiente de aplicaciones basadas en múltiples servicios desde un fichero.